



**Ciências
ULisboa**

DEPARTAMENTO INFORMÁTICO

Mestrado em Engenharia Informática

Projeto - Aplicação do NEAT ao jogo Racing Cars

Inteligência Artificial em Jogos

Students:

fc59810 - Afonso Almeida

fc59806 - João Nunes

Teacher:

Paulo Urbano

Junho 2026

Índice

1	Introdução	2
2	Arquitetura Geral e Simulação Física	3
2.1	Modularidade do Sistema e Desacoplamento	3
2.2	Modelo Cinemático do Veículo e Detecção de Colisões	3
2.3	Modelação de Incerteza (Injeção de Ruído Não Determinístico)	4
2.4	Robustez e Mecanismo de Inversão de Pista	4
3	Abordagem 1: Sistema de Navegação Reativa por Radares	5
3.1	Perceção do Ambiente (Inputs da Rede)	5
3.2	Atuação e Controlo (Outputs da Rede)	5
3.3	Função de Avaliação (<i>Fitness</i>)	5
4	Abordagem 2: Sistema de Navegação Espacial por <i>Waypoints</i>	7
4.1	Perceção Espacial (Inputs da Rede)	7
4.2	Atuação (Outputs da Rede)	7
4.3	Mecânica de Recompensa (Função de <i>Fitness</i>)	7
5	Avaliação Experimental e Comparação de Desempenho	9
5.1	Metodologia de Avaliação e Ambientes de Teste	9
5.2	Análise das Curvas de Aprendizagem (<i>Fitness</i>)	9
5.3	Complexidade Estrutural e Topologias de Rede	10
5.4	Discussão de Resultados e <i>Trade-offs</i>	11
6	Conclusão	12

1 Introdução

O desenvolvimento de agentes autônomos em videogames tem transitado de abordagens puramente determinísticas — como as árvores de decisão tradicionais e sensores estáticos — para modelos adaptativos baseados em Machine Learning. Neste contexto, o presente projeto explora a Neuro-Evolução através da aplicação do algoritmo NEAT (NeuroEvolution of Augmenting Topologies) ao jogo "Racing Cars", desenvolvido em Python (pygame).

Pertencente à classe dos TWEANNs (Topology and Weight Evolving Artificial Neural Networks), o NEAT destaca-se pela sua capacidade de evoluir simultaneamente os pesos sinápticos e a própria estrutura da rede neuronal. O algoritmo arranca com uma arquitetura minimalista (apenas nós de input e output) e complexifica a topologia ao longo das gerações, garantindo que a rede cresce apenas o estritamente necessário para resolver o problema.

O objetivo central deste trabalho consistiu em conceber, treinar e avaliar controladores neurais para os veículos do jogo, explorando dois paradigmas distintos de percepção do ambiente:

- **Condução Reativa (Radares):** O veículo utiliza feixes de detecção de distância (Raycasting) para inferir os limites da pista de forma dinâmica.
- **Navegação Espacial (Waypoints):** O veículo recebe apenas informação direcional e de distância relativa a uma sequência de pontos-alvo definidos na pista.

Para garantir que as redes desenvolvidas são robustas e não apenas memorizações exatas do circuito, o projeto incorporou também a simulação de incerteza do mundo físico (ruído nos atuadores) e experimentações com regras de condução alternativas (e.g., inversão do sentido da pista). No final, pretende-se não só demonstrar a viabilidade do NEAT na resolução do problema, mas também comparar a performance, o tempo de convergência e a complexidade topológica gerada por cada uma das abordagens.

2 Arquitetura Geral e Simulação Física

2.1 Modularidade do Sistema e Desacoplamento

De forma a garantir as boas práticas de engenharia de software e a eficiência computacional indispensável ao processo evolutivo, a arquitetura do projeto foi estruturada seguindo o princípio da separação de preocupações (Separation of Concerns).

O motor de jogo, a renderização gráfica e a física dos veículos foram inteiramente encapsulados no módulo autônomo `base_jogo.py`. Este desacoplamento radical garante que os scripts de treino bi-variantes (`Radares.ipynb` e `Waypoints.ipynb`) e o script de avaliação (`05_Visualizacao_e_Resultados.ipynb`) operem puramente ao nível da lógica da Inteligência Artificial. Desta forma, o algoritmo NEAT atua como um controlador abstrato que recebe um vetor de estados da simulação e devolve um vetor de ações, sem necessidade de conhecer os detalhes de implementação do ecossistema gráfico `pygame`.

2.2 Modelo Cinemático do Veículo e Detecção de Colisões

De forma a simular o comportamento do automóvel em pista, foi implementado um modelo cinemático bidimensional (2D) que rege os movimentos de translação e rotação sobre o plano do circuito. O estado físico instantâneo de cada veículo é caracterizado de forma unívoca por um vetor contendo as coordenadas cartesianas do centro de massa (x, y) , a velocidade escalar linear v e o ângulo de orientação θ (expresso em graus relativamente ao semieixo positivo das abcissas).

A atualização dinâmica do estado do agente a cada ciclo de computação (*frame*) obedece rigorosamente às seguintes formulações mecânicas:

1. **Aceleração e Resistência:** A velocidade escalar é atualizada linearmente em função da força de aceleração aplicada pela rede neuronal. É introduzida uma força de resistência passiva constante (simulando o atrito dos pneus e o arrasto aerodinâmico) que atenua a velocidade, garantindo a existência de uma velocidade terminal assintótica e impedindo divergências físicas na simulação.
2. **Atualização Posicional:** A projeção espacial do deslocamento translacional do veículo é obtida através da decomposição trigonométrica do vetor velocidade nos eixos coordenados globais, modelada pelas equações:

$$\Delta x = v \cdot \cos(\theta) \quad (1)$$

$$\Delta y = -v \cdot \sin(\theta) \quad (2)$$

A inversão de sinal no eixo das ordenadas (y) justifica-se pela convenção do referencial gráfico bidimensional padrão, cujo ponto de origem $(0, 0)$ se localiza no canto superior esquerdo do ecrã.

3. **Transformação de Coordenadas dos Vértices:** Para efeitos de renderização gráfica e para a validação precisa de colisões nas extremidades da viatura, as posições locais dos quatro vértices do retângulo que delimita o carro necessitam de ser projetadas para o espaço global. Este mapeamento geométrico

é realizado aplicando uma matriz de rotação bidimensional sobre os desvios diagonais locais ($\pm dx, \pm dy$) a partir do centro do carro. Com o intuito de otimizar o tempo de computação por geração, esta operação foi vetorizada recorrendo à biblioteca *NumPy*:

$$\begin{bmatrix} x_{\text{rot}} \\ y_{\text{rot}} \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \begin{bmatrix} \pm dx \\ \pm dy \end{bmatrix} \quad (3)$$

onde $\alpha = \theta \cdot \frac{\pi}{180}$ representa o ângulo de orientação convertido para radianos.

O algoritmo de detecção de colisões opera ao nível do píxel, salvaguardando a precisão contra aproximações poligonais grosseiras. Utilizando o subsistema de máscaras do *Pygame* (`pygame.mask`), é efetuada uma operação de interseção lógica binária (*AND bitwise*) entre a máscara geométrica dos vértices transformados do carro e a máscara rígida das bordas do circuito (`TRACK_BORDER_MASK`). A detecção de qualquer ponto de sobreposição resulta na interrupção imediata da simulação para esse espécime específico e no acionamento das penalizações da função de *fitness*.

2.3 Modelação de Incerteza (Injeção de Ruído Não Determinístico)

Para dar cumprimento ao Requisito 3.3 do enunciado e aproximar a simulação das restrições de um ambiente físico real (onde imperam imperfeições mecânicas e flutuações de aderência), foi introduzido um modelo estocástico nos atuadores do veículo.

Antes de as decisões da rede neuronal — aceleração (*throttle*) e rotação (*steering*) — alterarem o estado físico do carro, é injetado ruído através de perturbações aleatórias contínuas geradas pela função `uniform(-ruído, ruído)`. Esta injeção de ruído não determinístico impede que as redes neuronais criem soluções de "sobreajuste" (*overfitting*) baseadas puramente na memorização estrita de temporizações. Em vez disso, obriga o NEAT a selecionar redes que desenvolvam uma verdadeira capacidade de controlo em malha fechada, capazes de corrigir pequenos desvios de trajetória em tempo real.

2.4 Robustez e Mecanismo de Inversão de Pista

Com o intuito de testar a capacidade de generalização dos controladores e evitar que a evolução convergisse para soluções que apenas funcionavam virando para um dos lados, foi desenhado um mecanismo adaptativo de inversão de pista (Requisito 3.4).

Através da configuração da flag global `INVERTER_PISTA`, o simulador altera dinamicamente:

- As coordenadas de início (*spawn point*) do veículo na grelha de partida.
- O ângulo de orientação inicial do veículo, rodando-o a 180°.
- A ordenação sequencial dos objetivos (no caso da variante por waypoints).

3 Abordagem 1: Sistema de Navegação Reativa por Radares

A primeira abordagem desenvolvida para o controlo autónomo do veículo baseia-se num paradigma puramente reativo, onde o agente toma decisões com base na percepção imediata dos limites físicos do circuito. Para o efeito, foi implementado um sistema de sensores de distância simulados através da técnica de *Raycasting*, cumprindo o estabelecido no Requisito 3.2.

3.1 Percepção do Ambiente (Inputs da Rede)

Para mapear o ambiente circundante, o veículo projeta cinco raios virtuais (*radars*) a partir do seu centro de massa. Estes raios estão distribuídos simetricamente de forma a cobrir o campo de visão frontal do carro, operando nos ângulos relativos de -60° , -30° , 0° , 30° e 60° .

O algoritmo de *Raycasting* avança iterativamente ao longo da linha de cada raio, verificando a colisão com a máscara dos limites da pista (`TRACK_BORDER_MASK`). O alcance máximo de cada radar foi limitado a 200 píxeis para otimizar o processamento. De forma a garantir a estabilidade numérica e facilitar a convergência dos pesos da rede neuronal durante o treino NEAT, as distâncias em píxeis obtidas por cada sensor são normalizadas para o intervalo contínuo $[0.0, 1.0]$, onde 1.0 representa a ausência de obstáculos no raio máximo de alcance, e valores próximos de 0.0 indicam colisão iminente. Estes cinco valores normalizados constituem o vetor de entrada (*input*) da rede neuronal.

3.2 Atuação e Controlo (Outputs da Rede)

Com base na leitura dos sensores, a topologia evoluída pelo NEAT processa a informação e debita dois valores contínuos nos seus nós de saída (*outputs*), mapeados através de funções de ativação adequadas:

- **Aceleração (*Throttle*):** Um valor no intervalo $[-1, 1]$ que dita a força aplicada no acelerador ou na travagem/marcha-atrás.
- **Direção (*Steering*):** Um valor no intervalo $[-1, 1]$ que controla o ângulo de viragem das rodas, permitindo curvar à esquerda (valores negativos) ou à direita (valores positivos).

Conforme descrito na arquitetura do simulador (Requisito 3.3), as saídas da rede sofrem uma injeção de ruído não determinístico antes de atuarem sobre o modelo cinemático, testando a capacidade de o controlador manter o veículo estável face a perturbações.

3.3 Função de Avaliação (*Fitness*)

O sucesso do algoritmo NEAT depende criticamente do desenho da função de *fitness*, que guia o processo evolutivo. Para esta variante, a heurística de recompensa foi desenhada para encorajar a progressão rápida e segura ao longo da pista.

A cada *frame*, a função de *fitness* de um genoma é incrementada num valor proporcional à sua velocidade instantânea ($fitness += v$). Este mecanismo premeia agentes que completam o percurso mais rapidamente e penaliza indiretamente travagens desnecessárias. Foram também implementadas condições de término rigorosas para otimizar o tempo de simulação:

1. **Colisão:** Se os vértices do carro intersectarem as bordas da pista, o genoma é imediatamente desativado.
2. **Inatividade/Inércia:** Se a velocidade do carro permanecer próxima de zero nos primeiros 3 segundos (ou noutro momento crítico), a simulação do agente é terminada, evitando que o tempo computacional seja desperdiçado com redes que não aprenderam a acelerar ou que ficaram presas.

4 Abordagem 2: Sistema de Navegação Espacial por *Waypoints*

A segunda iteração de controlo desenvolvida no âmbito deste projeto foca-se num paradigma de navegação espacial orientada a objetivos (*Goal-Oriented Navigation*), de forma a dar resposta ao Requisito 3.1. Nesta variante, os sensores de deteção física (radares) foram removidos. O agente autónomo desconhece a geometria da pista de forma explícita, sendo guiado exclusivamente através de uma sequência predefinida de coordenadas bidimensionais, designadas por *Waypoints*, que delineiam a trajetória ideal do circuito.

4.1 Perceção Espacial (Inputs da Rede)

A abstração do problema para um modelo de perseguição de alvos sequenciais permitiu simplificar drasticamente a camada de entrada (*input layer*) da rede neuronal evoluída pelo NEAT. O controlador passa a processar apenas a relação espacial entre o veículo e o seu alvo imediato através de dois parâmetros vitais, rigorosamente normalizados para estabilizar o gradiente de aprendizagem:

- **Distância Euclidiana:** A distância linear entre o centro de massa do veículo e as coordenadas do *waypoint* atual. Para evitar a injeção de valores arbitrariamente grandes na rede, esta métrica é normalizada (dividindo pelo comprimento diagonal máximo do ecrã, por exemplo), mapeando o valor final para o intervalo contínuo $[0.0, 1.0]$.
- **Diferença Angular (*Delta Angle*):** O desvio relativo entre o ângulo de orientação atual do carro (θ) e o vetor direcional que aponta para o alvo. Este ângulo é normalizado para o intervalo $[-1.0, 1.0]$, onde 0.0 indica um alinhamento perfeito (o carro aponta diretamente para o *waypoint*), eliminando descontinuidades angulares (como a transição de 360° para 0°).

4.2 Atuação (Outputs da Rede)

À semelhança da arquitetura reativa por radares, os nós de saída da rede neuronal geram valores de atuação contínuos no intervalo $[-1, 1]$ para a aceleração (*Throttle*) e direção (*Steering*). A injeção de ruído não determinístico pré-atuação (Requisito 3.3) mantém-se ativa nesta versão. A persistência desta incerteza mecânica obriga o agente a aprender mecânicas de micro-correção na direção para se manter alinhado com o *waypoint*, evitando que o NEAT evolua agentes que operem unicamente em malha aberta (memorização cega de manobras).

4.3 Mecânica de Recompensa (Função de *Fitness*)

A alteração do paradigma de perceção exigiu uma reestruturação da heurística de avaliação. A função de *fitness* foi adaptada para premiar o progresso sequencial do agente em direção à meta:

1. **Recolha de *Waypoints*:** Foi definido um raio geométrico de tolerância de 25 píxeis em torno de cada ponto-alvo. Quando a distância euclidiana do veículo é igual ou inferior a este limiar, o *waypoint* é considerado "recolhido", o alvo transita iterativamente para o ponto seguinte no vetor de posições, e o genoma recebe um bônus substancial (200 pontos de *fitness*).
2. **Bonificação Final:** A conclusão do trajeto e o corte da meta resultam num grande prêmio de aptidão final, sendo o tempo decorrido registado.
3. **Penalização e Poda Evolutiva:** Mantêm-se os critérios rigorosos de exclusão. A colisão com as bordas da pista (`TRACK_BORDER_MASK`) continua a resultar na morte instantânea da rede. Em paralelo, espécimes incapazes de gerar velocidade motriz suficiente nos primeiros instantes da simulação sofrem rescisão prematura, canalizando os recursos computacionais da geração para agentes viáveis.

5 Avaliação Experimental e Comparação de Desempenho

Nesta secção, apresenta-se a análise comparativa detalhada dos resultados obtidos no treino das duas abordagens estruturadas (Radares e *Waypoints*). O processo de avaliação foi centralizado no *script* complementar `05_Visualizacao_e_Resultados.ipynb`, o qual carrega os históricos de evolução guardados em formato binário (`stats_radares.pkl` e `stats_waypoints.pkl`) e as respectivas estruturas de rede validadas pelo ecossistema NEAT.

5.1 Metodologia de Avaliação e Ambientes de Teste

Ambos os controladores foram submetidos a um processo evolutivo sob as mesmas condições base de simulação, utilizando uma dimensão populacional fixa e taxas equiparáveis de mutação estrutural (adição/remoção de nós e conexões). O critério de paragem automática foi definido pelo alcance do *fitness* de corte associado à conclusão bem-sucedida do circuito dentro do tempo limite. Adicionalmente, a robustez final dos melhores espécimes de cada geração foi aferida aplicando o mecanismo de inversão de pista (INVERTER_PISTA), garantindo que os resultados refletem competências reais de condução e não apenas fenómenos de sobre-ajuste (*overfitting*).

5.2 Análise das Curvas de Aprendizagem (*Fitness*)

A convergência do algoritmo NEAT pode ser escrutinada através do comportamento das métricas de *fitness* máximo e *fitness* médio ao longo das gerações, conforme ilustrado na Figura 1.

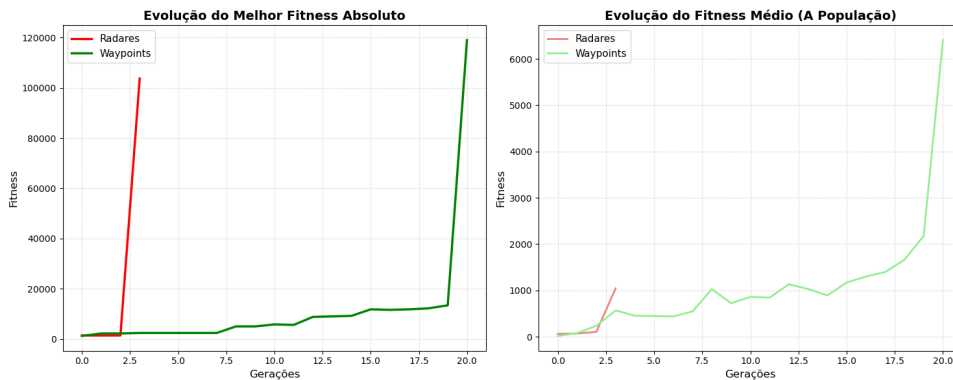


Figure 1: Evolução do Fitness Absoluto (Melhor Genoma) e Fitness Médio da População para as abordagens de Radares e Waypoints.

O estudo comparativo dos dados revela perfis evolutivos nitidamente distintos:

- **Curva de Radares:** Exibe uma progressão inicial mais lenta e marcada por patamares estáveis prolongados. Isto deve-se à maior dimensionalidade do espaço de estados (5 *inputs*), o que exige que o NEAT teste múltiplas permutações de pesos sinápticos antes de correlacionar eficazmente as distâncias

dos raios com a prevenção de colisões. No entanto, após a descoberta de uma topologia basilar estável, a curva de *fitness* máximo apresenta um crescimento acentuado.

- **Curva de *Waypoints*:** Demonstra uma taxa de convergência substancialmente superior nas gerações iniciais. Sendo os *inputs* compostos por métricas de erro pré-calculadas (distância euclidiana e diferença angular direta), o gradiente de aprendizagem torna-se mais linear, permitindo que a população atinja pontuações elevadas num menor número de iterações geracionais.

O comportamento do *fitness* médio da população reforça este diagnóstico. Na abordagem por *waypoints*, o *fitness* médio acompanha de forma mais próxima a evolução do melhor espécime, o que indica que as mutações favoráveis se propagam rapidamente por especiação. Na abordagem por radares, o *fitness* médio permanece mais baixo e oscilatório, fruto da eliminação agressiva de agentes que colidem nas curvas mais apertadas da pista.

5.3 Complexidade Estrutural e Topologias de Rede

Uma das maiores vantagens do algoritmo NEAT reside na otimização topológica guiada pelo princípio da parcimónia (crescimento minimalista). Recorrendo à biblioteca *Graphviz* integrada no módulo `visualize.py`, procedeu-se à extração visual das redes neuronais dos melhores genomas de cada variante.

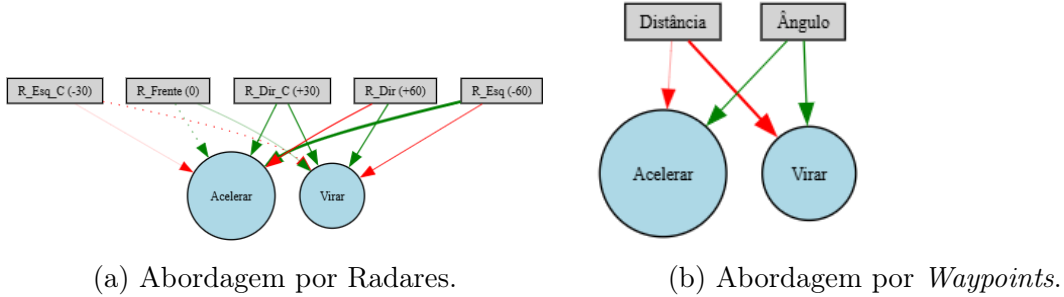


Figure 2: Topologias das redes neuronais dos controladores vencedores obtidos através do algoritmo NEAT.

A análise das estruturas anatómicas das redes (Figura 2) fundamenta matematicamente a diferença de desempenho computacional:

- A rede vencedora da variante de **Radares** desenvolveu uma estrutura mais densa, contendo múltiplos nós ocultos (*hidden nodes*) e um número elevado de conexões ativas. Esta complexidade justifica-se pela necessidade de mapear interações não lineares complexas (como, por exemplo, o equilíbrio entre a leitura do radar frontal e dos radares oblíquos a 30° para antecipar uma trajetória de viragem).
- A rede associada à condução por ***Waypoints*** estabilizou com uma configuração minimalista, operando em larga escala com ligações diretas entre as

entradas e as saídas, com pouca ou nenhuma necessidade de nós ocultos adicionais. O desvio angular normalizado atua como um gatilho direto sobre o nó de saída da direção (*steering*), traduzindo-se numa heurística simples de implementar e executar.

5.4 Discussão de Resultados e *Trade-offs*

Os dados de treino consolidados expõem um claro compromisso de engenharia (*trade-off*) entre as duas metodologias.

A abordagem por *waypoints* provou ser altamente eficiente no que toca ao custo computacional de treino: exige menos memória, menos gerações e converge para soluções funcionais de forma muito mais célere. Contudo, do ponto de vista prático da Inteligência Artificial em Jogos, apresenta uma vulnerabilidade crítica: a rigidez ambiental. Uma vez que o agente depende de coordenadas estáticas fixas, qualquer alteração na topologia da pista ou reposicionamento de curvas obrigaria a um mapeamento manual exaustivo de novos *waypoints* e a um re-treino total do modelo.

Por outro lado, a abordagem baseada em radares, apesar de acarretar um custo computacional de treino superior e originar redes estruturalmente mais pesadas, oferece uma capacidade de generalização e resiliência marcadamente superior. Sendo um sistema puramente reativo à geometria local envolvente, o controlador baseado em radares exhibe potencial para navegar com sucesso em pistas inteiramente novas e dinâmicas sem necessidade de qualquer alteração no seu código ou arquitetura sináptica.

6 Conclusão

O projeto desenvolvido permitiu uma exploração prática e fundamentada da Neuro-Evolução, validando a eficácia do algoritmo NEAT na resolução de problemas de controlo de agentes autónomos em ambientes de simulação física. A comparação direta entre a navegação por sensores reativos (Radares) e a navegação orientada a objetivos (Waypoints) permitiu obter conclusões valiosas sobre o comportamento das Redes Neurais Artificiais (ANNs) e o impacto das escolhas de *input* na convergência do algoritmo.

Através deste trabalho, foi possível verificar que o sucesso do NEAT não depende apenas da capacidade computacional, mas principalmente do desenho da função de *fitness* e da qualidade da normalização dos dados de entrada. A abordagem por *waypoints* destacou-se pela sua eficiência e rapidez de convergência, demonstrando ser a solução ideal para ambientes onde a trajetória ideal é conhecida a priori. Em contrapartida, a abordagem por radares, embora mais exigente em termos de treino e complexidade topológica, provou ser a arquitetura mais robusta e versátil, capaz de atuar em malha fechada sem depender de coordenadas externas pré-definidas.

A introdução de fatores como o ruído estocástico nos atuadores e o mecanismo de inversão de pista foram cruciais para evitar fenómenos de sobre-ajuste, garantindo que as soluções evoluídas possuissem uma capacidade real de generalização. Em suma, o projeto atingiu todos os objetivos propostos, permitindo a compreensão profunda de como estruturas neuronais evolutivas conseguem aprender a resolver problemas complexos através de um processo de ensaio e erro, espelhando mecanismos adaptativos observados na biologia.

Como trabalho futuro, sugere-se a expansão deste simulador para ambientes com obstáculos dinâmicos móveis ou a implementação de uma arquitetura de *Deep Reinforcement Learning* para comparação de performance contra a neuro-evolução pura, testando se a capacidade de generalização poderia ser ainda mais refinada em cenários de incerteza extrema.